Practical Guide to Building Reliable Document Q&A Systems

This guide describes a practical lifecycle for a small document question-answering system. A reliable implementation begins with a well-defined document collection and an ingestion process that can be repeated. It should load supported files, extract text, preserve metadata, split content into useful chunks, create embeddings in batches, and persist a vector index. The querying process should be separate so that users can ask multiple questions without reprocessing every source document.

The first design decision is the scope of the knowledge base. Documents should be relevant, sufficiently detailed, and legally suitable for inclusion. The developer should inspect each document for text quality, duplicate versions, confidential information, and extraction problems. Digital PDFs are usually straightforward to parse, while image-only PDFs may need optical character recognition. A system should not silently index an empty document. It should report which files were loaded and which could not be processed.

A chunk is the unit of text that the retriever searches. Very large chunks may contain useful evidence mixed with unrelated material, while very small chunks can lose context. Fixed-size splitting with overlap is a reasonable baseline because it is easy to explain and reproduce. When the loader preserves page-level documents, each chunk can inherit the source filename and page number. This allows the user to verify where an answer originated. The README should state the chosen chunk size and overlap and explain that these values may need tuning for other collections.

Embedding models transform text into vectors that can be compared by semantic similarity. The same embedding model should be used for document chunks and user questions. Calls should be batched to avoid inefficient one-at-a-time inference. For a small project, a lightweight sentence-transformer model is practical and can run locally. Model selection should consider language, domain, hardware, and the quality of retrieval on representative questions rather than relying only on popularity.

A vector index such as FAISS supports nearest-neighbour search over dense vectors. Persistence usually involves saving the index and the metadata that maps vector positions to text chunks. Those files must remain aligned. The application should validate the saved files when loading them and provide an actionable message if indexing has not yet been run. If the documents or embedding model change, the index should be rebuilt so old vectors are not mixed with new content.

The retrieval stage converts the user's question into an embedding and retrieves the top-k nearest chunks. The value of k should be configurable. More retrieved passages can provide wider coverage but may introduce distracting text, while too few may omit a key detail. Developers should inspect retrieved passages when an answer is wrong. If evidence is missing from the retrieval results, the problem is likely in ingestion, chunking, embeddings, or search settings. If evidence is present but the answer is wrong, generation instructions or context formatting may be responsible.

The generation stage should receive a clearly delimited set of passages and an explicit instruction to answer only from those passages. If the context does not contain the answer, the model should say that the available documents do not provide enough information. This instruction reduces unsupported answers but does not eliminate all errors. A robust application should also handle empty retrieval results and should show the source passages separately from the generated response.

Citations should be controlled by the application, not invented by the language model. A source label should identify the actual filename and page or section from metadata. The user should be able to inspect the retrieved text that supports an answer. This makes the system more transparent and helps a reviewer evaluate both retrieval and generation. If the model refers to source identifiers, the application should ensure that those identifiers correspond to the context it actually supplied.

A good test plan includes direct questions, paraphrased questions, multi-document questions, and questions that are not answerable from the collection. Tests should check whether the expected evidence is retrieved,

whether the answer captures the key facts, and whether unanswerable questions are handled cautiously. Operational tests should include restarting the app, missing API configuration, malformed files, an empty data directory, and an absent index. Errors should be informative but must not expose API keys.

A beginner project should prioritize clarity over complexity. Separate indexing from querying, use understandable data structures, keep source metadata, document the architecture, and show the pipeline in a live recording. The README should give exact commands for installation, configuration, indexing, launching the interface, and running tests. A reviewer should be able to clone the repository and follow the instructions without needing undocumented help.

No retrieval system is perfectly reliable. Embeddings may miss relevant passages, tables may be extracted poorly, and a language model may misinterpret the evidence it receives. These limitations should be stated honestly. For high-impact topics, the application should not be presented as a replacement for qualified professional judgment. The goal of a small RAG assignment is to demonstrate a complete and inspectable pipeline, not to claim that generated answers are always correct.